

Skill-04 code:

```
GNU nano 8.7.1 Skill-04.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 100
#define MAX_TEXT 100
#define MAX_ARGS 50

typedef enum {
    TOKEN_WORD,
    TOKEN_PIPE,
    TOKEN_INPUT,
    TOKEN_OUTPUT,
    TOKEN_APPEND,
    TOKEN_BACKGROUND,
    TOKEN_SEMICOLON
} TokenType;

typedef struct {
    TokenType type;
    char value[MAX_TEXT];
} Token;

typedef struct {
    char command[MAX_TEXT];
    char *args[MAX_ARGS];
    int arg_count;

    char input_file[MAX_TEXT];
    char output_file[MAX_TEXT];

    int append;
    int background;
} Command;

const char *tokenTypeName(TokenType type)
{
    switch (type) {
        case TOKEN_WORD:
            return "WORD";
        case TOKEN_PIPE:
            return "PIPE";
        case TOKEN_INPUT:
            return "INPUT";
        case TOKEN_OUTPUT:
            return "OUTPUT";
        case TOKEN_APPEND:
            return "APPEND";
        case TOKEN_BACKGROUND:
            return "BACKGROUND";
        case TOKEN_SEMICOLON:
            return "SEMICOLON";
        default:
            return "UNKNOWN";
    }
}
```

```
GNU nano 8.7.1 Skill-04.c
        case TOKEN_BACKGROUND:
            return "BACKGROUND";
        case TOKEN_SEMICOLON:
            return "SEMICOLON";
        default:
            return "UNKNOWN";
    }
}

/* Tokenize input */
int tokenize(const char *input, Token tokens[])
{
    int count = 0;
    int i = 0;

    while (input[i] != '\0' && count < MAX_TOKENS - 1) {
        if (isspace((unsigned char)input[i])) {
            i++;
            continue;
        }

        /* Pipe */
        if (input[i] == '|') {
            tokens[count].type = TOKEN_PIPE;
            strcpy(tokens[count].value, "|");
            count++;
            i++;
        }

        /* Input redirection */
        else if (input[i] == '<') {
            tokens[count].type = TOKEN_INPUT;
            strcpy(tokens[count].value, "<");
            count++;
            i++;
        }

        /* Output / append */
        else if (input[i] == '>') {
            if (input[i + 1] == '>') {
                tokens[count].type = TOKEN_APPEND;
                strcpy(tokens[count].value, ">>");
                count++;
                i += 2;
            }
            else {
                tokens[count].type = TOKEN_OUTPUT;
            }
        }
    }
}
```

```

        strcpy(tokens[count].value, ">");
        count++;
        i++;
    }
}

/* Background */
else if (input[i] == '&') {
    tokens[count].type = TOKEN_BACKGROUND;
    strcpy(tokens[count].value, "&");
    count++;
    i++;
}

/* Semicolon */
else if (input[i] == ';') {
    tokens[count].type = TOKEN_SEMICOLON;
    strcpy(tokens[count].value, ";");
    count++;
    i++;
}

/* Normal word */
else {
    int j = 0;

    tokens[count].type = TOKEN_WORD;

    while (input[i] != '\0' &&
           !isspace((unsigned char)input[i]) &&
           input[i] != '|' &&
           input[i] != '<' &&
           input[i] != '>' &&
           input[i] != '&' &&
           input[i] != ';') {

        if (j < MAX_TEXT - 1) {
            tokens[count].value[j++] = input[i];
        }

        i++;
    }

    tokens[count].value[j] = '\0';
    count++;
}
}

return count;

```

```

/* Display tokens */
void printTokens(Token tokens[], int count)
{
    printf("\n===== TOKENS =====\n");

    for (int i = 0; i < count; i++) {
        printf("%2d : %-12s : %s\n",
               i + 1,
               tokenTypeName(tokens[i].type),
               tokens[i].value);
    }
}

/* Parse tokens into commands */
int parseCommands(Token tokens[], int count, Command commands[])
{
    int command_count = 0;
    int arg_count = 0;

    memset(commands, 0, sizeof(Command) * MAX_TOKENS);

    for (int i = 0; i < count; i++) {
        /* Normal word */
        if (tokens[i].type == TOKEN_WORD) {
            if (commands[command_count].command[0] == '\0') {
                strcpy(commands[command_count].command,
                       tokens[i].value);
            }

            if (arg_count < MAX_ARGS - 1) {
                commands[command_count].args[arg_count] =
                    malloc(strlen(tokens[i].value) + 1);

                if (commands[command_count].args[arg_count] == NULL) {
                    printf("Memory allocation failed\n");
                    return -1;
                }

                strcpy(commands[command_count].args[arg_count],
                       tokens[i].value);

                arg_count++;
                commands[command_count].arg_count = arg_count;
            }
        }
    }
}

```

```

/* Pipe */
else if (tokens[i].type == TOKEN_PIPE) {
    commands[command_count].args[arg_count] = NULL;
    command_count++;
    arg_count = 0;
}

/* Input redirection */
else if (tokens[i].type == TOKEN_INPUT) {
    if (i + 1 < count &&
        tokens[i + 1].type == TOKEN_WORD) {
        strcpy(commands[command_count].input_file,
            tokens[i + 1].value);
        i++;
    }
}

/* Output redirection */
else if (tokens[i].type == TOKEN_OUTPUT) {
    if (i + 1 < count &&
        tokens[i + 1].type == TOKEN_WORD) {
        strcpy(commands[command_count].output_file,
            tokens[i + 1].value);
        commands[command_count].append = 0;
        i++;
    }
}

/* Append redirection */
else if (tokens[i].type == TOKEN_APPEND) {
    if (i + 1 < count &&
        tokens[i + 1].type == TOKEN_WORD) {
        strcpy(commands[command_count].output_file,
            tokens[i + 1].value);
        commands[command_count].append = 1;
    }
}

```

```

        i++;
    }
}

/* Background */
else if (tokens[i].type == TOKEN_BACKGROUND) {
    commands[command_count].background = 1;
}

/* Semicolon */
else if (tokens[i].type == TOKEN_SEMICOLON) {
    commands[command_count].args[arg_count] = NULL;
    command_count++;
    arg_count = 0;
}

commands[command_count].args[arg_count] = NULL;
return command_count + 1;
}

/* Display parsed commands */
void printCommands(Command commands[], int count)
{
    printf("\n===== PARSED COMMANDS =====\n");
    for (int i = 0; i < count; i++) {
        if (commands[i].command[0] == '\0') {
            continue;
        }
        printf("\nCommand %d\n", i + 1);
        printf("Command      : %s\n",
            commands[i].command);
        printf("Arguments    : ");
        for (int j = 0; j < commands[i].arg_count; j++) {
            printf("%s ", commands[i].args[j]);
        }
        printf("\n");
    }
}

```

```

GNU nano 8.7.1 Skill-04.c
    if (commands[i].input_file[0] != '\0') {
        printf("Input file : %s\n",
               commands[i].input_file);
    }

    if (commands[i].output_file[0] != '\0') {
        printf("Output file : %s\n",
               commands[i].output_file);

        if (commands[i].append) {
            printf("Output mode : APPEND\n");
        }
        else {
            printf("Output mode : WRITE\n");
        }
    }

    if (commands[i].background) {
        printf("Background : YES\n");
    }
    else {
        printf("Background : NO\n");
    }
}

/* Free dynamically allocated memory */
void freeCommands(Command commands[], int count)
{
    for (int i = 0; i < count; i++) {
        for (int j = 0;
             j < commands[i].arg_count;
             j++) {
            free(commands[i].args[j]);
        }
    }
}

int main()
{
    char input[MAX_TEXT + 3];

    Token tokens[MAX_TOKENS];
    Command commands[MAX_TOKENS];

    printf("=====\n");
    printf("          COMMAND PARSER DEMO          \n");

```

```

printf("=====\n");
printf("          COMMAND PARSER DEMO          \n");
printf("=====\n");

printf("\nEnter a command: ");

if (fgets(input, sizeof(input), stdin) == NULL) {
    printf("Input error\n");
    return 1;
}

input[strcspn(input, "\n")] = '\0';

/* Tokenize */
int token_count = tokenize(input, tokens);

/* Print tokens */
printTokens(tokens, token_count);

/* Parse commands */
int command_count =
    parseCommands(tokens, token_count, commands);

if (command_count < 0) {
    return 1;
}

/* Print parsed commands */
printCommands(commands, command_count);

/* Free memory */
freeCommands(commands, command_count);

printf("\n=====\n");
printf("          Parser completed successfully          \n");
printf("=====\n");

return 0;

```

Output:

```

uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-04$ nano Skill-04.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-04$ gcc Skill-04.c -o Skill-04
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-04$ ./Skill-04

```

```

=====
COMMAND PARSER DEMO
=====

Enter a command: ls || pwd

===== TOKENS =====
1 : WORD      : ls
2 : PIPE      : |
3 : PIPE      : |
4 : WORD      : pwd

===== PARSED COMMANDS =====

Command 1
Command      : ls
Arguments    : ls
Background   : NO

Command 3
Command      : pwd
Arguments    : pwd
Background   : NO

=====
Parser completed successfully
=====

```

Syntax error:

```

uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-04$ ./skill4_parser

```

```

=====
COMMAND PARSER DEMO
=====

Enter a command: valgrind --leak-check=full ./skill4_parser

===== TOKENS =====
1 : WORD      : valgrind
2 : WORD      : --leak-check=full
3 : WORD      : ./skill4_parser

===== PARSED COMMANDS =====

Command 1
Command      : valgrind
Arguments    : valgrind --leak-check=full ./skill4_parser
Background   : NO

=====
Parser completed successfully
=====

```